

Subprograme partea 1

Wednesday, April 29, 2026 6:55 PM

Subprogram = colectie de tipuri de date, variabile, instructiuni care indeplinesc o anumita sarcina => un SCOP

- Poate fi folosit in alte parti din cod prin **apelare**
- In cazul in care avem cod identic, cu acelasi scop in main putem sa il punem intr-o functie
- Acestea impart problema in subprobleme mai mici
- Se gasesc usor erorile

Subprogramele pot fi de 2 tipuri:

1. Proceduri: instructiuni de sine statatoare care indeplinesc o sarcina si nu returneaza/da un rezultat mai departe ex: citirea de la tastura a unui sir de valori, afisarea etc.
 2. Functii: - determina un rezultat, o anumita valoare, pornind de la datele de intrare => valoare returnata de functie
 - Acestea fata de proceduri pot fi apelate ca operanzi in expresii ($a = \text{functie}(a) + a;$), pot fi apelate ca sa se afiseze valoare returnata ($\text{cout} << \text{functie}(a);$) si in alte structuri (decizie si repetitive)
- in C/C++ exista doar subprograme de tip functie, pt proceduri avem o forma particulara a functiilor

Structura functiei care returneaza ceva

```
Tip_returnat nume_functie(tip1 parametru1, tip2 parametru2, ..., tipn parametrun)  ANTETUL FUNCTIEI
{
    //instructiuni de cod
    return ceva;  CORPUL FUNCTIEI
}
```

ANTET

Nume_functie = modul in care denumim functia si o recunoaste programul (Atentie este unic, in unele cazuri speciale poate fi acelasi*!) + sa fie sugestiv

Tip_returnat = ce tip de variabila va fi ce dam ca rezultat (int, float, double, long long etc)

Lista parametri/argumente = ceea ce primesc din partea programului care apeleaza/cheama functia si care vor participa pe parcursul executiei in instructiuni

- Tip1, tip2, ..., tip n sunt tipurile de date ale parametrilor (int, float, bool, char etc)
- Parametru1, ..., parametrun = numele parametrilor (fara restrictii, dar sa fie cat mai general numele

Suma a doua numere intregi:

```
int suma(int a, int b)
{
    int s;
    s=a+b;
    c=a-b;
    return s;
}
```

CORPUL FUNCTIEI

-exista instructiuni + variabile

-variabile din functii sunt locale, dupa terminarea functiei, pierdem valoarea lor (spre exemplu in functia suma se pierde valoarea lui c)

-cuvantul "return" este sugestiv folosit si reprezinta faptul ca se va da rezultatul si functia si-a terminat executia (in particular, se vor sterge din memorie variabilele locale si instructiunile functiei)

- return poate aparea in mai multe locuri in functie si semnaleaza ca s-a indeplinit ceva ca sa se incheie functia

Ex: calculul impartirii a doua nr intregi pozitive+ partea intreaga a rezultatului

```
int impartire_intreaga(int a, int b)
{
    If(b==0)
        return -1;
    int c;
    c=a/b;
    return c;
}
```

- Nu ne trebuie un else, fiindca daca se indeplineste conditia, functia va returna/ne va da -1 si se va incheia,

restul instructiunilor ramase nu se vor executa

-in antetul functiei pot exista 0, 1 sau mai multi parametri

-pt 0 parametrii:

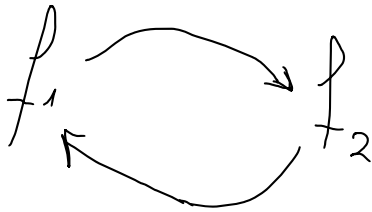
```
int x; //variabila globala
int y;
int sum()
{
    return x+y;
}
```

OBS: RETURN VALOARE(VARIABILA); sau RETURN EXPRESIE(CARE DETERMINA O SINGURA VALOARE);

Ce inseamna apelarea unei functii?

Apelarea reprezinta chemarea functiei ca sa execute ceva intr-o anumita parte a codului

Apelarea se poate face in main, dar si in alte functii (ATENTIE: NU apelez functia f1 in f2 si dupa f2 in f1)



Ordinea in care sunt scrise functiile conteaza (daca f1 este scrisa prima in fisier, poate fi apelata de celelalte functii desub ea)

Pentru a scrie sub main se poate face urmatorul artificiu: scriem antetul functiei cu ; la final cu promisiunea ca undeva in cod vom implementa functia respectiva

Structura functiei care nu returneaza ceva - tipuri speciale de functii care inlocuiesc procedurile pt C/C++

```
void nume_functie(tip1 parametru1, tip2 parametru2, ..., tipn parametrun) ANTETUL FUNCTIEI
{
    //instructiuni de cod CORPUL FUNCTIEI
}
```

-poate sa aiba sau nu instructiunea **return;** care semnifica oprire fortata in locul acela din functie pentru ca se indeplineste o conditie

- Poate avea 0,1, sau mai multi parametrii

```
void citire_elem_vector(int n, int v[])
{
    for(int i=1;i<=n;i++)
        cin>>v[i];
}
```

f(f(..f()...)) - se calculeaza de la interior spre exterior

```
#include <iostream>
#include <cmath>
using namespace std;
```

```
void citire(int v[], int& n)
{
    cin>>n;
    for(int i=1;i<=n;i++)
        cin>>v[i];
}
```

```
void afisare(int m, int a[])
{
    cout<<m<<endl;
    for(int i=1;i<=m;i++)
        cout<<a[i]<<" ";
}
```

```

void citire_mat(int b[][26], int& linie, int& c)
{
    cin>>linie>>c;
    for(int i=1;i<=linie;i++)
        for(int j=1;j<=c;j++)
            cin>>b[i][j];
}

void afisare_mat(int l, int a[][20], int c)
{
    cout<<l<<" "<<c<<endl;
    for(int i=1;i<=l;i++)
    {
        for(int j=1;j<=c;j++)
            cout<<a[i][j]<<" ";
        cout<<endl;
    }
}

int suma(int a, int b=0)
{
    int s = a+b;
    return s;
}

int main()
{
    /*int n; /// va lua o valoare random
    cout<<n<<endl;
    int v[100]; /// va fi initializat cu valori random
    citire(v,n);
    afisare(n,v);*/

    /*int l, c;
    int a[25][26];
    citire_mat(a,l,c);
    afisare_mat(l, a, c);*/
    int a=5;
    cout<<suma(3,2)<<endl;

    //cout<<suma(a)<<endl;
    return 0;
}

```